

Digital Design & Computer Arch.

Lecture 15c: Load-Store Handling in Out-of-Order Execution

Prof. Onur Mutlu

ETH Zürich

Spring 2025

15 April 2025

Handling Out-of-Order Execution of Loads and Stores

Registers versus Memory

- So far, we considered mainly registers as part of state

- What about memory?

Memory dependencies are not at all time it needs to be executed to find the dependencies.

↑

- What are the fundamental differences between registers and memory?

- Register dependences known statically – memory dependences determined dynamically
- Register state is small – memory state is large (addresses are very large)
- Register state is not visible to other threads/processes – memory state is shared between threads/processes (in a shared memory multiprocessor)

In a multithreaded problem, threads don't share registers whereas they can access memory (memory is shared b/w them)

We know reg addr at 30 stage $\therefore$ we can handle them by register renaming. But memory

Memory Dependence Handling (I)

addr is not known

- Need to obey memory dependences in an out-of-order machine
 - and need to do so while providing high performance
- Observation and Problem: Memory address is not known until a load/store executes
 - ↑ $\therefore$ problem
 - if instr. j is loading value from mem addr A then there can be instr. i storing value (writing at mem addr A) and renaming is difficult.
- Corollary 1: Renaming memory addresses is difficult
- Corollary 2: Determining dependence or independence of loads/stores has to be handled *after* their (partial) execution
 - ↑ i.e. after calc the addresses
- Corollary 3: (When a load/store has its address ready, there may be older/younger stores/loads with unknown addresses in the machine)

illy viceversa.

i.e.

more serious

st addr, rs

ld addr, re

problem

if same addr

Memory Dependence Handling (II)

this is the least
scalable part
of ooo
engine.

- When do you schedule a load instruction in an OOO engine?
 - Problem: A younger load can have its address ready before an older store's address is known (or)
 - Known as the memory disambiguation problem or the unknown address problem.
↳ i.e. Store load.
- Approaches
 - (Only stalls, no use). (not much helpful) (no perf gain)
 - **Conservative:** Stall the load until all previous stores have computed their addresses (or even retired from the machine)
 - **Aggressive:** Assume load is independent of unknown-address stores and schedule the load right away. (Assume load addr is indep of store addr & continue)
 - kind of speculative
□ **Intelligent:** Predict (with a more sophisticated predictor) if the load is dependent on any unknown address store
∴ predictor is required in this case

Handling of Store-Load Dependences

- **A load's dependence status is not known** until all previous store addresses are available.
- How does the OOO engine detect dependence of a load instruction on a previous store?
 - Option 1: [Wait until all previous stores committed] (no need to check for address match)
 - Option 2: { Keep a list of pending stores in a store buffer and check whether load address matches a previous store address } ?
- How does the OOO engine treat the scheduling of a load instruction wrt previous stores?
 - Option 1: Assume load dependent on all previous stores
 - Option 2: Assume load independent of all previous stores
 - Option 3: Predict the dependence of a load on an outstanding store

Memory Disambiguation (I)

■ Option 1: Assume load is dependent on all previous stores

+ No need for recovery

Then have to wait until all stores are done.

-- Too conservative: delays independent loads unnecessarily (Load's takes longer & stores time)

■ Option 2: Assume load is independent of all previous stores

+ Simple and can be common case: no delay for independent loads

-- Requires recovery and re-execution of load and dependents on misprediction
logic to identify that we're wrong (In case if we're wrong we need to recover & re-execute the load)

■ Option 3: Predict the dependence of a load on an outstanding store (Intelligent option)

∴ we can learn from previous executions.

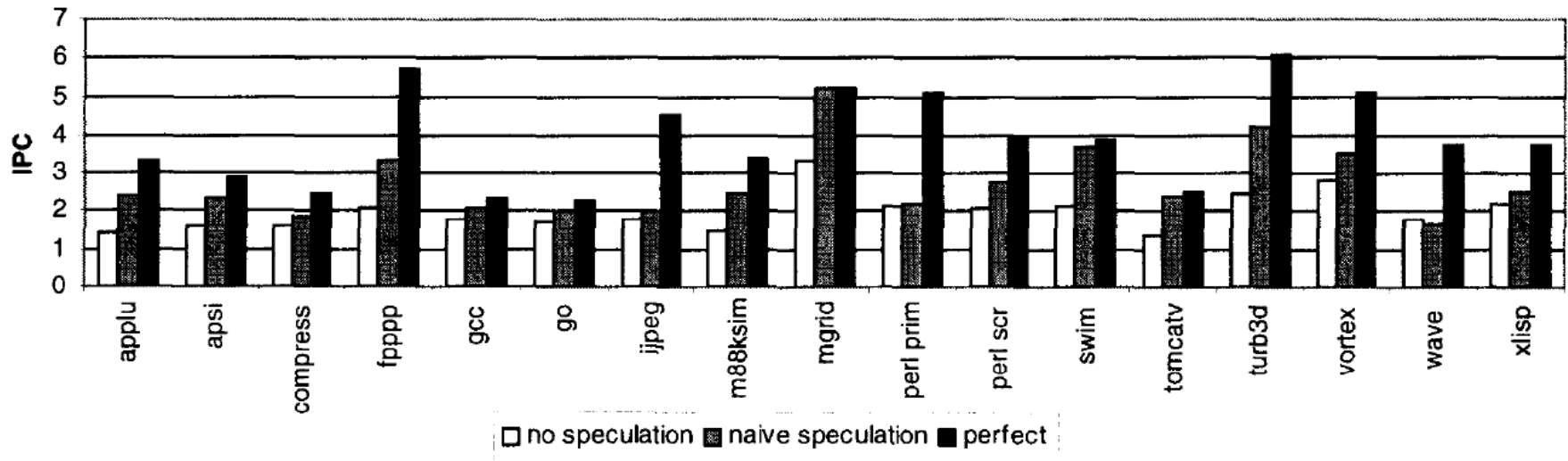
+ More accurate. Load store dependences persist over time

-- Still requires recovery/re-execution on misprediction

- Alpha 21264 : Initially assume load independent, delay loads found to be dependent
- Moshovos et al., “**Dynamic speculation and synchronization of data dependences**,” ISCA 1997.
- Chrysos and Emer, “**Memory Dependence Prediction Using Store Sets**,” ISCA 1998.

Memory Disambiguation (II)

- Chrysos and Emer, “**Memory Dependence Prediction Using Store Sets**,” ISCA 1998.



- Predicting store-load dependences important for performance
- Simple predictors (based on past history) can achieve most of the potential performance

Data Forwarding Between Stores and Loads

- We cannot update memory out of program order
→ (Need to buffer all store and load instructions in instruction window)
is one as any instr commit in order.
- Even if we **know** all addresses of past stores when we generate the address of a load, two questions still remain:
logic to see if addr gen by cur load is dependent on any previous addr gen by earlier stores.
 1. How do we check whether or not it is dependent on a store ✓
 2. How do we forward data to the load if it is dependent on a store
- Modern processors use a LQ (load queue) and a SQ for this
To do above things
 - Can be combined or separate between loads and stores (A)
 - (A load searches the SQ after it computes its address) Why? *Once that we found that addr gen by load is dep on store then how do you forward the data from store to the load.*
 - A store searches the LQ after it computes its address. Why? *To see if there is any store that*
2
To see if a load is dep on it. (useful for point-1) *once store gen's addr it compares with load to see if dependent/not.* } *we are dep-on.*

Out-of-Order Completion of Memory Ops

- When a store instruction finishes execution, it writes its address and data in its reorder buffer entry (or SQ entry)

- When a later load instruction generates its address, it:
 - searches the SQ with its address
 - accesses memory with its address
 - receives the value from the youngest older instruction that wrote to that address (either from ROB or memory)

if no matching is found / all older stores committed then fetches the value directly by accessing memory by the addr it generated.

But if mult stores match then take the youngest of the older stores.

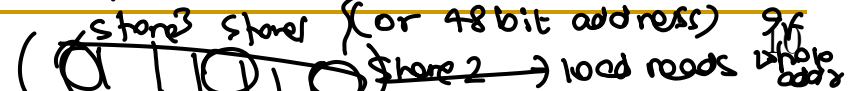
- This is a complicated "search logic" implemented as a Content Addressable Memory

∴ we may have to compare all addr in that load addr's

- Content is "memory address" (but also need *size* and *age*)
- Called **store-to-load forwarding logic**

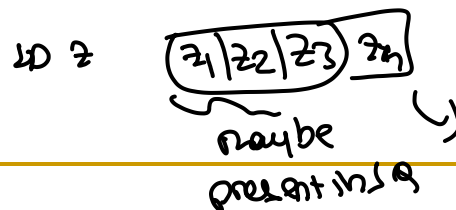
(Even though we compare addr it may not be suff bc 2, the addr from which load) 64 bit address

reading can be written at multi-places by diff stores

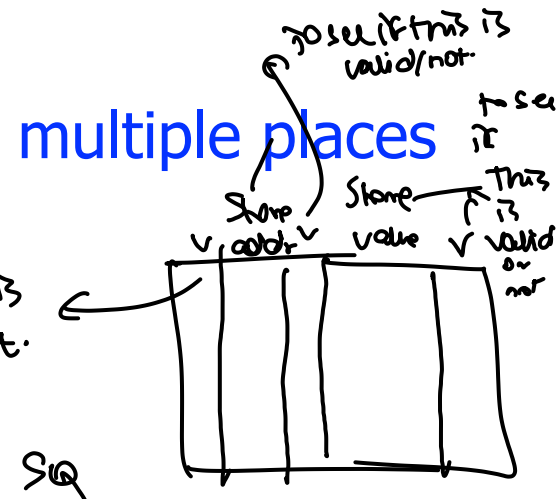


Store-Load Forwarding Complexity

- **Content Addressable Search** (based on Load Address) ✓
- **Range Search** (based on Address and Size of both the Load and earlier Stores) (Bcz addr of load may be partially written by some stores)
- **Age-Based Search** (for last written values)
 ↳ to get younger addr from old stores.
- **Load data can come from a combination of multiple places**
 - ❑ One or more stores in the Store Buffer (SQ)
 - ❑ Memory/cache



If not present → need to access memory to get the latest value from them.



CYCLE —

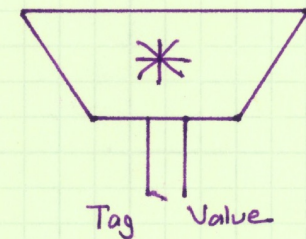
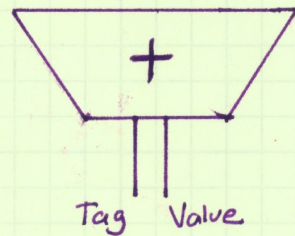
MUL R1, R2 → R3
 ADD R3, R4 → R5
 ADD R2, R6 → R7
 ADD R8, R9 → R10
 MUL R7, R10 → R11
 ADD R5, R11 → R5

Register Alias Table

	V	Tag	Value
R1			
R2			
R3			
R4			
R5			
R6			
R7			
R8			
R9			
R10			
R11			

	SOURCE 1			SOURCE 2		
	V	Tag	Value	V	Tag	Value
a						
b						
c						
d						

	SOURCE 1			SOURCE 2		
	V	Tag	Value	V	Tag	Value
x						
y						
z						
t						



Digital Design & Computer Arch.

Lecture 15c: Load-Store Handling in Out-of-Order Execution

Prof. Onur Mutlu

ETH Zürich

Spring 2025

15 April 2025